

MAST32001 Computational Statistics - Lecture 2

Numerical Linear Algebra

Klaus Nordhausen

University of Helsinki

WS 2026

Today

1. Why linear algebra is central to computational statistics
2. Linear systems and matrix decompositions
3. Conditioning and numerical stability
4. Least squares as a statistical example
5. R experiments with stable and unstable computations

Why numerical linear algebra?

Linear algebra everywhere in statistics

Many statistical computations reduce to matrix operations:

- ▶ least squares and generalized linear models;
- ▶ covariance matrices;
- ▶ principal component analysis;
- ▶ multivariate normal models;
- ▶ optimization;
- ▶ Gaussian processes;
- ▶ state-space models;
- ▶ MCMC and Bayesian computation.

The mathematics may be expressed compactly, but the numerical algorithm matters.

The central computational problem

A fundamental problem is to solve

$$Ax = b$$

for x .

Mathematically, if A is invertible,

$$x = A^{-1}b.$$

Computationally, however, we usually **do not form** A^{-1} .

We solve the system directly.

Why not compute the inverse?

Computing an explicit inverse can:

- ▶ require more work;
- ▶ introduce additional rounding error;
- ▶ use more memory;
- ▶ hide useful matrix structure.

In R:

```
solve(A, b)
```

is preferable to

```
solve(A) %*% b
```

when the goal is only to solve $Ax = b$.

Matrix norms and conditioning

Measuring size: vector and matrix norms

For a vector $x = (x_1, \dots, x_p)^\top$, the Euclidean norm

$$\|x\|_2 = \sqrt{x_1^2 + \dots + x_p^2}$$

is simply its **length**.

But what should the “size” of a matrix A mean?

A matrix transforms a vector x into a new vector Ax . We can therefore ask:

How large can $\|Ax\|_2$ become relative to $\|x\|_2$?

This leads to the **matrix 2-norm**

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}$$

It is the **largest multiplication of length** that A can produce.

What does this mean geometrically?

Consider all vectors of length one:

$$\|x\|_2 = 1.$$

They form a unit circle (or a unit sphere in higher dimensions).

After multiplication by A ,

$$x \longmapsto Ax,$$

the unit circle generally becomes an **ellipse**.

- The longest semi-axis has length

$$\sigma_{\max}(A) = \|A\|_2.$$

- The shortest semi-axis has length

$$\sigma_{\min}(A).$$

Thus the singular values describe how much A changes lengths in its most and least amplified directions.

Matrix norms in practice

The matrix 2-norm has an important characterization:

$$\|A\|_2 = \sigma_{\max}(A),$$

the largest singular value of A .

Other useful norms include

$$\|A\|_1 = \max_j \sum_i |a_{ij}|$$

and

$$\|A\|_\infty = \max_i \sum_j |a_{ij}|.$$

In R, `norm(A, type = "2")`, `"1"`, `"I"`, and `"F"` provide common matrix norms.

Condition number

For an invertible matrix A , the **condition number** is defined as

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

It measures how sensitive the solution of a linear system can be to small perturbations in the input.

The condition number depends on the choice of matrix norm.

For the **matrix 2-norm**,

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

- ▶ $\kappa(A) \geq 1$.
- ▶ A value close to 1 indicates a **well-conditioned** problem.
- ▶ A large value indicates an **ill-conditioned** problem.

A large condition number means that small perturbations in the data can lead to much larger changes in the solution.

Condition number and eigenvalues

For a general matrix, the 2-norm condition number is a ratio of **singular values**, not eigenvalues:

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

If A is **symmetric positive definite**, its singular values equal its positive eigenvalues. Then

$$\kappa_2(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}.$$

So the eigenvalue ratio is valid in this important special case, but not for an arbitrary matrix.

In base R, `kappa()` estimates a condition number; `kappa(A, exact = TRUE)` computes the 2-norm condition number from singular values.

Conditioning is a property of the problem

An important distinction:

- ▶ **conditioning** describes sensitivity of the mathematical problem;
- ▶ **stability** describes the behaviour of the numerical algorithm.

A good algorithm cannot make an inherently ill-conditioned problem well-conditioned.

But a poor algorithm can make a well-conditioned problem behave badly.

Condition numbers in R

```
A1 <- matrix(c(2, 0, 0, 1), 2, 2)
A2 <- matrix(c(1, 1, 1, 1 + 1e-10), 2, 2)
```

```
kappa(A1)
```

```
## [1] 1.666667
```

```
kappa(A2)
```

```
## [1] 39999822098
```

The second matrix is close to singular.

Small perturbation, large effect

```
b1 <- c(2, 2)
b2 <- b1 + c(0, 1e-10)
```

```
solve(A2, b1)
```

```
## [1] 2 0
```

```
solve(A2, b2)
```

```
## [1] 1 1
```

A tiny change in b can lead to a visible change in the solution when the system is ill-conditioned.

Active part: conditioning

R TASK: can you trust the coefficients?

Work with your neighbour.

```
set.seed(20260812)
n <- 100
x1 <- rnorm(n)
x2 <- x1 + 1e-6 * rnorm(n)
X_task <- cbind(1, x1, x2)
y_task <- 1 + x1 + x2 + rnorm(n, sd = 0.1)
```

1. Compare `kappa(X_task)` and `kappa(crossprod(X_task))`.
2. Estimate the coefficients using `qr.solve(X_task, y_task)`.
3. Change only the first response:

```
y2 <- y_task
y2[1] <- y2[1] + 0.01
```

4. Estimate the coefficients again.
5. Compare both the **coefficients** and the **fitted values**.

What do you observe?

Solution: can you trust the coefficients?

```
kappa(X_task)
```

```
## [1] 2870827
```

```
kappa(crossprod(X_task))
```

```
## [1] 7.6241e+12
```

```
b1_task <- qr.solve(X_task, y_task)
```

```
y2_task <- y_task
```

```
y2_task[1] <- y2_task[1] + 0.01
```

```
b2_task <- qr.solve(X_task, y2_task)
```

```
cbind(  
  original = b1_task,  
  perturbed = b2_task,  
  difference = b2_task - b1_task  
)
```

Solution: can you trust the coefficients?

```
fit1_task <- as.vector(X_task %*% b1_task)
fit2_task <- as.vector(X_task %*% b2_task)

c(
  max_coefficient_change = max(abs(b2_task - b1_task)),
  max_fitted_value_change = max(abs(fit2_task - fit1_task))
)

## max_coefficient_change max_fitted_value_change
## 1.065699e+01 4.539195e-04
```

What happened?

The columns x_1 and x_2 are almost linearly dependent.

Therefore X is ill-conditioned, and forming

$$X^\top X$$

makes the conditioning substantially worse:

$$\kappa_2(X^\top X) = \kappa_2(X)^2.$$

The data contain much more information about the **combined effect** of x_1 and x_2 than about how that effect should be divided between the two coefficients.

Thus individual coefficients can be sensitive while fitted values remain comparatively stable.

Matrix decompositions

LU decomposition

For a general square matrix, Gaussian elimination leads to an LU factorization

$$PA = LU,$$

where:

- ▶ P is a permutation matrix;
- ▶ L is lower triangular;
- ▶ U is upper triangular.

Triangular systems can then be solved efficiently.

Why pivoting?

Gaussian elimination repeatedly uses a **pivot** to eliminate entries below it.

If the pivot is zero, elimination cannot continue without reordering. If it is very small, division by that pivot can create very large intermediate numbers and amplify rounding error.

Partial pivoting swaps rows so that the largest available absolute entry in the current column is used as the pivot.

This leads to

$$PA = LU,$$

where P records the row exchanges.

Pivoting: a small example

Consider

$$A = \begin{pmatrix} 10^{-10} & 1 \\ 1 & 1 \end{pmatrix}.$$

Using 10^{-10} as the first pivot requires a multiplier of 10^{10} .

Partial pivoting exchanges the rows first and uses 1 as the pivot instead.

The mathematical linear system is unchanged; the elimination path is numerically much safer.

More sophisticated strategies can also pivot columns, but partial row pivoting is the standard choice for general dense LU factorization.

Cholesky decomposition

If A is symmetric positive definite,

$$A = LL^{\top}$$

for a lower triangular matrix L .

Cholesky decomposition is especially important for:

- ▶ covariance matrices;
- ▶ multivariate normal simulation;
- ▶ Gaussian likelihoods;
- ▶ weighted least squares.

It is efficient and exploits matrix structure.

QR decomposition

For an $n \times p$ matrix X ,

$$X = QR,$$

where Q has orthonormal columns and R is upper triangular.

QR is central to numerical least squares because orthogonal transformations preserve Euclidean lengths and are numerically well behaved.

Singular value decomposition (SVD): dimensions

Let $X \in \mathbb{R}^{n \times p}$ and let $r = \text{rank}(X) \leq \min(n, p)$. A full SVD can be written as

$$X = UDV^\top,$$

with $U \in \mathbb{R}^{n \times n}$, $D \in \mathbb{R}^{n \times p}$ and $V \in \mathbb{R}^{p \times p}$. The columns of U and V are orthonormal.

The matrix D is “diagonal” (rectangular if $n \neq p$):

$$D = \begin{pmatrix} \sigma_1 & & & \\ & \ddots & & \\ & & \sigma_r & \\ & & & 0 \end{pmatrix}, \quad \sigma_1 \geq \cdots \geq \sigma_r > 0.$$

The σ_i are the **singular values** of X .

Only r singular values are nonzero; all remaining entries of D are zero.
Thus r tells us how many directions are actually needed to represent X .

This motivates the **thin SVD**, which keeps only these r directions.

Thin SVD

In statistical computing we often use the **thin** or **economy-size** SVD.

If $k = \min(n, p)$,

$$X = U_k D_k V_k^\top,$$

where

$$U_k : n \times k, \quad D_k : k \times k, \quad V_k : p \times k.$$

If only the nonzero singular directions are retained, k can be replaced by the rank r .

The diagonal entries

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$$

of D are the singular values.

What the SVD tells us

The SVD reveals:

- ▶ rank and near rank-deficiency;
- ▶ the strongest and weakest directions in the data;
- ▶ the matrix 2-norm:

$$\|X\|_2 = \sigma_1;$$

- ▶ the 2-norm condition number:

$$\kappa_2(X) = \sigma_1/\sigma_k;$$

- ▶ principal-component directions.

Small singular values identify directions in which the matrix contains little independent information.

Active part: choosing a decomposition

TASK: which decomposition?

Work with your neighbour.

For each problem, choose the most natural decomposition from:

LU

Cholesky

QR

SVD

1. Solve a general square system $Ax = b$.
2. Solve a least-squares problem

$$\min_{\beta} \|y - X\beta\|_2^2.$$

3. Repeatedly solve systems involving a symmetric positive-definite covariance matrix.
4. Determine whether a data matrix is numerically rank deficient.

For each choice, give **one reason**.

Solution: which decomposition?

Problem	Natural choice	Why?
General $Ax = b$	LU	General-purpose factorization for square systems
Least squares	QR	Avoids forming $X^T X$; orthogonal transformations are stable
Positive-definite covariance matrix	Cholesky	Exploits symmetry and positive definiteness; efficient
Numerical rank	SVD	Small singular values directly reveal near rank-deficiency

These are not absolute rules. Several decompositions can solve the same mathematical problem.

The point is to exploit **matrix structure** and choose an algorithm suited to the numerical question.

One problem, several possible tools

For example, least squares can also be studied using the SVD.

Why might we prefer SVD over QR?

- ▶ the design is close to rank deficient;
- ▶ we want to inspect singular values;
- ▶ we need a pseudoinverse;
- ▶ we want explicit information about weakly identified directions.

So the question is often not

“Which decomposition is mathematically possible?”

but rather

“Which decomposition exposes the structure I need?”

Least squares

Linear regression

Consider the linear model

$$y = X\beta + \varepsilon.$$

Ordinary least squares minimizes

$$\|y - X\beta\|_2^2.$$

The familiar mathematical solution is

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

This formula should not automatically be interpreted as an algorithm.

Normal equations

Differentiating the least-squares objective gives

$$X^{\top} X \hat{\beta} = X^{\top} y.$$

One computational approach is therefore to solve the normal equations.

But forming $X^{\top} X$ changes the numerical conditioning.

Squaring the condition number

Approximately,

$$\kappa(X^\top X) = \kappa(X)^2.$$

Thus a moderately ill-conditioned design matrix can become a severely ill-conditioned normal-equation system.

This is a major reason QR is generally preferred for least squares.

Three ways to compute least squares

We can compare:

1. explicit inverse:

$$(X^{\top} X)^{-1} X^{\top} y;$$

2. solve the normal equations:

$$\text{solve}(X^{\top} X, X^{\top} y);$$

3. QR decomposition.

R's `lm()` also uses numerically appropriate linear algebra rather than explicitly forming the inverse.

R example: well-conditioned regression

Simulate a simple problem

```
n <- 1000
x1 <- rnorm(n)
x2 <- rnorm(n)

X <- cbind(1, x1, x2)
beta <- c(1, 2, -1)
y <- as.vector(X %*% beta + rnorm(n, sd = 0.5))

kappa(X)
```

```
## [1] 1.028577
```

Compare coefficient calculations

```
XtX <- crossprod(X)
Xty <- crossprod(X, y)

b_inverse <- solve(XtX) %*% Xty
b_normal <- solve(XtX, Xty)
b_qr <- qr.solve(X, y)
b_lm <- coef(lm(y ~ x1 + x2))

cbind(
  inverse = as.vector(b_inverse),
  normal = as.vector(b_normal),
  QR = as.vector(b_qr),
  lm = as.vector(b_lm)
)
```

##		inverse	normal	QR	lm
##	[1,]	1.008401	1.008401	1.008401	1.008401
##	[2,]	2.000835	2.000835	2.000835	2.000835
##	[3,]	-1.025513	-1.025513	-1.025513	-1.025513

R example: nearly collinear predictors

Construct an ill-conditioned design

```
set.seed(20260812)

n <- 1000
x1 <- rnorm(n)
x2 <- x1 + rnorm(n, sd = 1e-8)

X_bad <- cbind(1, x1, x2)
beta <- c(1, 2, -1)
y_bad <- as.vector(X_bad %*% beta + rnorm(n, sd = 0.5))

kappa(X_bad)
```

```
## [1] 196367253
```

```
kappa(t(X_bad) %*% X_bad)
```

```
## [1] 2.273009e+15
```

What does collinearity mean here?

The columns x_1 and x_2 are almost identical.

The data contain strong information about combinations such as

$$\beta_1 + \beta_2,$$

but weak information about separating β_1 from β_2 .

This is not merely a numerical issue: the statistical problem itself is poorly conditioned.

Compare methods

```
b_normal_bad <- try(  
  solve(crossprod(X_bad), crossprod(X_bad, y_bad)),  
  silent = TRUE  
)  
  
b_qr_bad <- qr.solve(X_bad, y_bad, tol = 1e-12)  
  
fit_bad <- lm(y_bad ~ x1 + x2)
```

Compare methods

```
b_normal_bad
```

```
##           [,1]  
##      9.928328e-01  
## x1  6.152250e+04  
## x2 -6.152153e+04
```

```
b_qr_bad
```

```
##           x1           x2  
## 9.926998e-01 1.274787e+06 -1.274786e+06
```

```
coef(fit_bad)
```

```
## (Intercept)           x1           x2  
## 0.9928396 0.9727567 NA
```

Different algorithms may expose the near rank-deficiency differently.

Singular values

```
sv <- svd(X_bad)$d
```

```
sv
```

```
## [1] 4.481075e+01 3.162277e+01 2.283039e-07
```

```
max(sv) / min(sv)
```

```
## [1] 196276786
```

A very small singular value indicates a direction in parameter space that is weakly identified by the design.

Perturb the response

```
y_bad2 <- y_bad
y_bad2[1] <- y_bad2[1] + 1e-7

b1 <- qr.solve(X_bad, y_bad, tol = 1e-12)
b2 <- qr.solve(X_bad, y_bad2, tol = 1e-12)

cbind(original = b1, perturbed = b2, difference = b2 - b1)
```

```
##          original      perturbed      difference
##      9.926998e-01  9.926998e-01  1.003175e-10
## x1  1.274787e+06  1.274787e+06 -4.175810e-03
## x2 -1.274786e+06 -1.274786e+06  4.175810e-03
```

A tiny perturbation can produce a much larger change in individual coefficients.

Predictions may still be stable

```
pred1 <- as.vector(X_bad %*% b1)
pred2 <- as.vector(X_bad %*% b2)

c(
  max_coefficient_change = max(abs(b2 - b1)),
  max_prediction_change = max(abs(pred2 - pred1))
)
```

```
## max_coefficient_change  max_prediction_change
##           4.175810e-03           9.313226e-10
```

Unstable coefficients do not necessarily imply equally unstable fitted values.

This distinction matters when interpreting regression models.

Useful numerical linear algebra in R

Useful base R functions

Some functions worth knowing:

- ▶ `crossprod(X)` computes $X^\top X$;
- ▶ `crossprod(X, Y)` computes $X^\top Y$;
- ▶ `tcrossprod(X)` computes $XX^\top$;
- ▶ `tcrossprod(X, Y)` computes $XY^\top$;
- ▶ `solve(A, b)` solves $Ax = b$;
- ▶ `qr()`, `qr.solve()` for QR methods;
- ▶ `chol()` and `chol2inv()` for positive-definite matrices;
- ▶ `svd()` for singular value decomposition;
- ▶ `eigen()` for eigenvalues/eigenvectors;
- ▶ `kappa()` and `rcond()` for conditioning diagnostics;
- ▶ `norm()` for matrix norms.

Why `crossprod()` and `tcrossprod()`?

Instead of

```
t(X) %*% X
```

write

```
crossprod(X)
```

and instead of

```
X %*% t(X)
```

write

```
tcrossprod(X)
```

These express the intended operation directly and can avoid explicitly constructing the transposed matrix.

They are particularly common in statistical computing.

More useful functions

Also useful:

- ▶ `backsolve()` and `forwardsolve()` for triangular systems;
- ▶ `diag()` for extracting/constructing diagonals;
- ▶ `sweep()` for row/column-wise centring and scaling;
- ▶ `scale()` for standardization;
- ▶ `determinant(..., logarithm = TRUE)` for stable log-determinants;
- ▶ `unname()` / `as.vector()` when presentation requires removal of matrix attributes.

For large sparse problems, the `Matrix` package provides sparse matrix classes and decompositions.

Practical numerical linear algebra

Avoid explicit inverses

Instead of

```
solve(A) %*% b
```

use

```
solve(A, b)
```

Instead of implementing least squares as

```
solve(t(X) %*% X) %*% t(X) %*% y
```

prefer QR-based routines such as `lm()`, `lm.fit()`, or `qr.solve()`.

Exploit structure

Choose algorithms that use what you know about the matrix:

- ▶ symmetric positive definite $\rightarrow$ Cholesky;
- ▶ least squares $\rightarrow$ QR;
- ▶ rank or conditioning questions $\rightarrow$ SVD;
- ▶ sparse matrices $\rightarrow$ sparse methods;
- ▶ repeated solves with the same matrix $\rightarrow$ factorize once.

Cross-products also appear in covariance

For a centred $n \times p$ data matrix X_c ,

$$S = \frac{1}{n-1} X_c^\top X_c.$$

In R:

```
Xc <- scale(X, center = TRUE, scale = FALSE)
S <- crossprod(Xc) / (nrow(Xc) - 1)
```

If observations were stored in columns instead, `tcrossprod()` would give the analogous row cross-product efficiently.

Scaling matters

Predictors on very different scales can worsen conditioning.

```
Z <- cbind(  
  rnorm(1000, sd = 1e-6),  
  rnorm(1000, sd = 1e6)  
)  
  
c(  
  raw = kappa(Z),  
  scaled = kappa(scale(Z))  
)
```

```
##           raw          scaled  
## 9.898851e+11 1.146311e+00
```

Centering and scaling can improve numerical behaviour and often improve interpretation.

Residual checks

A computed solution $\hat{x}$ to $Ax = b$ can be checked through the residual

$$r = b - A\hat{x}.$$

A small residual is useful, but it does not by itself guarantee that $\hat{x}$ is close to the true solution when the problem is ill-conditioned.

Both residual size and conditioning matter.

Numerical linear algebra in statistics

When a statistical computation behaves strangely, ask:

- ▶ Is the matrix nearly singular?
- ▶ Are predictors nearly collinear?
- ▶ Are variables on radically different scales?
- ▶ Am I forming an inverse unnecessarily?
- ▶ Am I forming $X^T X$ unnecessarily?
- ▶ Is there a decomposition designed for this problem?

Summary

What you should remember

- ▶ Linear algebra is a core computational language of statistics.
- ▶ Conditioning describes sensitivity of the problem.
- ▶ Stability describes behaviour of the algorithm.
- ▶ Explicit matrix inversion is usually unnecessary.
- ▶ QR is a strong default for least squares.
- ▶ Forming $X^T X$ squares the condition number.
- ▶ SVD is useful for diagnosing rank and near-singularity.
- ▶ Statistical identifiability and numerical stability are related, but not the same thing.